

GRAPH THEORY – GROUP HOMEWORK

EULERIAN TOUR

Informatics Engineering – Institut Teknologi Sepuluh Nopember

Group: 3

Members:

1. Dzulfiqar Rafi'ussunnah - 5025251011
2. Padhang Abiyu Fikri - 5025251014
3. Aditya Lingga Mardika - 5025251158
4. Muhammad Faris Alfarrel - 5025251002

Course: Graph Theory

1. Introduction

The purpose of this homework is to solve the CSES Mail Delivery problem (Problem 1691), which can be modeled as an Eulerian Circuit problem. The graph represents a city's road network, where crossings are vertices and streets are edges. A mail carrier must start at the post office, visit every street exactly once, and return to the post office.

1.1 Problem Statement

The problem requires finding a route that starts at vertex 1, traverses every undirected edge exactly once, and returns to vertex 1. If such a route does not exist, the program must output IMPOSSIBLE

1.2 Objective

- Implement Fleury's Algorithm to find an Eulerian circuit.
- Implement Hierholzer's Algorithm to find an Eulerian circuit.
- Implement a third Eulerian algorithm using Backtracking.
- Test all three implementations using the CSES Mail Delivery problem.
- Compare the approaches and their time complexity.

2. Fleury's Algorithm

2.1 Algorithm Overview

Fleury's Algorithm is an algorithm used to construct an Eulerian circuit or Eulerian trail by traversing edges one at a time while avoiding bridges whenever possible. The main principle of the algorithm is never cross a bridge unless there is no alternative edge. This strategy prevents the graph from becoming disconnected before all edges have been visited.

2.2 Algorithm Procedure

1. Start from vertex 1, which represents the post office.
2. Check that every vertex has an even degree. If any vertex has an odd degree, print IMPOSSIBLE.
3. Verify that all vertices containing edges are connected to vertex 1.
4. Store the graph using an adjacency list.
5. From the current vertex, examine the available edges.

6. If only one edge remains, traverse that edge.
7. Otherwise, check whether each candidate edge is a bridge.
8. Select a non-bridge edge whenever possible. If all remaining edges are bridges, select one of them.
9. Remove the selected edge from the graph and move to the next vertex.
10. Repeat the process until all edges have been traversed.
11. Verify that the route ends at vertex 1 and contains exactly $m + 1$ vertices.
12. Print the resulting Eulerian circuit.

2.3 Step-by-Step Process

For the sample input, Fleury's Algorithm starts from vertex 1. At each step, the algorithm chooses an edge that is not a bridge whenever another choice is available. Only when the remaining edge is a bridge does the algorithm traverse it. This guarantees that the graph remains connected until every edge has been visited.

Step	Current Route	Action
1	1	Start from vertex 1.
2	$1 \rightarrow 2$	Select a non-bridge edge from vertex 1.
3	$1 \rightarrow 2 \rightarrow 6$	Select a non-bridge edge from vertex 2.
4	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3$	Select a non-bridge edge from vertex 6.
5	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2$	Continue using a non-bridge edge.
6	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4$	Traverse another available non-bridge edge.
7	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5$	Continue traversal.
8	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3$	Continue traversal using the remaining valid edge.
9	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$	Traverse the final remaining edge and return to the starting vertex .

After all 8 edges have been traversed, the algorithm produces the Eulerian circuit:

$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$

This route starts and ends at vertex 1 and visits every street exactly once.

2.4 Implementation

Fleury's Algorithm was implemented using Python with an adjacency list represented by a set for each vertex, allowing edges to be removed efficiently during traversal. Before constructing the Eulerian circuit, the implementation verifies that every vertex has an even degree and that all vertices with edges are connected to the starting vertex.

To determine whether an edge is a bridge, the implementation temporarily ignores the selected edge and performs a graph traversal to check whether an alternative path still exists between its endpoints. If another path exists, the edge is not considered a bridge and can be selected.

Otherwise, the edge is treated as a bridge and is used only when no other alternative is available. A cache of previously identified bridges is also maintained to reduce repeated bridge checks and improve the overall performance of the implementation.

2.5 Execution and Result

The program was executed using the given sample input. The output below shows the Eulerian circuit generated by Fleury's Algorithm.

```
1 2
1 3
2 3
2 4
2 6
3 5
3 6
4 5
1 2 3 5 4 2 6 3 1
```

Task:	Mail Delivery		
Sender:	zuri		
Submission time:	2026-09-22 18:02:43 +0300		
Language:	Python3 (PyPy3)		
Status:	READY		
Result:	TIME LIMIT EXCEEDED		
Test results ▴			
test	verdict	time	
#1	ACCEPTED	0.04 s	»»
#2	ACCEPTED	0.04 s	»»
#3	ACCEPTED	0.04 s	»»
#4	ACCEPTED	0.04 s	»»
#5	ACCEPTED	0.04 s	»»
#6	TIME LIMIT EXCEEDED	--	»»
#7	TIME LIMIT EXCEEDED	--	»»
#8	TIME LIMIT EXCEEDED	--	»»
#9	ACCEPTED	0.37 s	»»
#10	ACCEPTED	0.31 s	»»
#11	ACCEPTED	0.04 s	»»
#12	ACCEPTED	0.04 s	»»
#13	TIME LIMIT EXCEEDED	--	»»
#14	ACCEPTED	0.25 s	»»
#15	ACCEPTED	0.25 s	»»
#16	ACCEPTED	0.04 s	»»

The output starts and ends at vertex 1, and every street is visited exactly once, satisfying the requirements of an Eulerian circuit. Although the implementation correctly follows the principle of Fleury's Algorithm, its running time is generally higher than Hierholzer's Algorithm because bridge detection may require repeated graph traversals during the execution. This is consistent with the theoretical complexity comparison between the two algorithms.

3. Hierholzer's Algorithm:

3.1 Algorithm Overview

Hierholzer's Algorithm is an algorithm used to find an **Eulerian circuit** in an undirected graph, where the route starts and ends at the same vertex and visits every edge exactly once. In the CSES Mail Delivery problem, the algorithm starts from vertex 1, which represents the post office, and repeatedly follows unused edges while storing the visited vertices in a stack. When the current vertex has no unused edges remaining, it is added to the final circuit and removed from the stack. This process continues until all edges have been used. An Eulerian circuit can be formed when all vertices with edges have an even degree and the required edges are connected to the starting vertex.

3.2 Algorithm Procedure

The procedure of Hierholzer's Algorithm is:

1. Start from vertex **1**, which represents the post office.
2. Check that every vertex has an even degree. If any vertex has an odd degree, print **IMPOSSIBLE**.
3. Store all edges in an adjacency list and assign a unique ID to each edge.
4. Use a stack to store the current route.
5. Select an unused edge from the current vertex and move to the next vertex.
6. Mark the selected edge as used.
7. If the current vertex has no unused edges remaining, remove it from the stack and add it to the circuit.
8. Continue the process until the stack becomes empty.
9. Reverse the resulting circuit because the vertices are added in reverse order.
10. Check that the circuit contains exactly $m + 1$ vertices and starts and ends at vertex 1.
11. Print the resulting Eulerian circuit.

3.3 Step-by-Step Process

For the sample input, the Iterative Euler Tour starts from vertex 1 and follows unused edges using an explicit stack. When the current vertex has no unused edges remaining, it is removed from the stack and added to the final path. The process continues until all edges have been traversed, producing an Eulerian circuit that returns to vertex 1.

Step	Current Route	Action
1	1	Start from vertex 1.
2	1 → 2	Follow an unused edge from 1 to 2.
3	1 → 2 → 6	Follow an unused edge from 2 to 6.
4	1 → 2 → 6 → 3	Follow an unused edge from 6 to 3.
5	1 → 2 → 6 → 3 → 2	Follow an unused edge from 3 to 2.
6	1 → 2 → 6 → 3 → 2 → 4	Follow an unused edge from 2 to 4.
7	1 → 2 → 6 → 3 → 2 → 4 → 5	Follow an unused edge from 4 to 5.
8	1 → 2 → 6 → 3 → 2 → 4 → 5 → 3	Follow an unused edge from 5 to 3.
9	1 → 2 → 6 → 3 → 2 → 4 → 5 → 3 → 1	Follow the remaining unused edge from 3 to 1.

After all **8 edges** have been used, the algorithm produces the Eulerian circuit:

1 → 2 → 6 → 3 → 2 → 4 → 5 → 3 → 1

This route starts and ends at vertex 1 and visits every street exactly once.

3.4 Implementation

Hierholzer's Algorithm was implemented using **Python** with an adjacency list to represent the graph. Each edge is assigned a unique ID so that the program can track whether an edge has already been used. A stack is used to store the current route, while a **used** array keeps track of visited edges. The implementation starts from vertex 1 and continues traversing unused edges until all edges have been processed. The resulting circuit is then reversed to obtain the correct order of the Eulerian circuit.

3.5 Execution and Result

The program was executed using the given sample input. The output below shows the Eulerian circuit generated by Hierholzer's Algorithm.

```
PS C:\Users\ASUS\Desktop\Final practicum Strukdat> python -u  
6 8  
1 2  
1 3  
2 3  
2 4  
2 6  
3 5  
3 6  
4 5  
1 2 3 5 4 2 6 3 1
```

Submission details

Task:	Mail Delivery
Sender:	RubySimp
Submission time:	2026-09-22 17:17:37 +0300
Language:	Python3 (PyPy3)
Status:	READY
Result:	ACCEPTED

Test results ▲

test	verdict	time	
#1	ACCEPTED	0.04 s	»»
#2	ACCEPTED	0.04 s	»»
#3	ACCEPTED	0.04 s	»»
#4	ACCEPTED	0.04 s	»»
#5	ACCEPTED	0.04 s	»»
#6	ACCEPTED	0.59 s	»»
#7	ACCEPTED	0.59 s	»»
#8	ACCEPTED	0.59 s	»»
#9	ACCEPTED	0.57 s	»»
#10	ACCEPTED	0.31 s	»»
#11	ACCEPTED	0.04 s	»»
#12	ACCEPTED	0.04 s	»»
#13	ACCEPTED	0.22 s	»»
#14	ACCEPTED	0.23 s	»»
#15	ACCEPTED	0.22 s	»»
#16	ACCEPTED	0.04 s	»»

The implementation successfully finds a valid Eulerian circuit for the given sample input. The resulting route is $1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$, which starts and ends at vertex 1 and uses each of the 8 edges exactly once.

4. Iterative Euler Tour Algorithm

4.1 Algorithm Overview

Instead of recursively calling a function for every edge, the algorithm stores the current traversal in an explicit stack. This is useful for large graphs because it avoids Python's recursion-depth limitation encountered by the previous Backtracking implementation. The algorithm repeatedly processes unused edges from the current vertex. When no unused edge remains at the current vertex, the vertex is removed from the stack and added to the final path.

4.2 Algorithm Procedure

1. Start from vertex 1.
2. Create an empty stack and push vertex 1.
3. While the stack is not empty:
 - Look at the vertex on top of the stack.
 - Find an unused edge incident to that vertex.
4. If an unused edge exists:
 - Mark the edge as used.
 - Determine the other endpoint.
 - Push the other endpoint onto the stack.
5. If no unused edge exists:
 - Remove the current vertex from the stack.
 - Add it to the final path.
6. Reverse the resulting path.
7. Check that the path contains exactly $E + 1$ vertices.
8. Check that the path starts and ends at vertex 1.

4.3 Step-by-Step Process

For the sample input, the Iterative Euler Tour begins at vertex 1 and traverses each unused edge using a stack. When no unused edge remains at the current vertex, the vertex is popped from the stack and added to the path. This process is repeated until all edges are used, resulting in a valid Eulerian circuit that starts and ends at vertex 1.

Step	Current Route	Action
1	1	Start from vertex 1.
2	1 → 3	Follow an unused edge from 1 to 2.
3	1 → 3 → 6	Follow an unused edge from 2 to 6.
4	1 → 3 → 6 → 2	Follow an unused edge from 6 to 3.
5	1 → 3 → 6 → 2 → 4	Follow an unused edge from 3 to 2.
6	1 → 3 → 6 → 2 → 4 → 5	Follow an unused edge from 2 to 4.
7	1 → 3 → 6 → 2 → 4 → 5 → 3	Follow an unused edge from 4 to 5.
8	1 → 3 → 6 → 2 → 4 → 5 → 3 → 2	Follow an unused edge from 5 to 3.
9	1 → 3 → 6 → 2 → 4 → 5 → 3 → 2 → 1	Follow the remaining unused edge from 3 to 1.

4.4 Implementation

The implementation uses an explicit stack to construct the Eulerian circuit iteratively. Each edge is assigned an edge ID and marked as used when traversed. If the current vertex still has an unused edge, the next vertex is pushed onto the stack. If no unused edge remains, the current vertex is popped from the stack and added to the final path. The resulting path is reversed at the end to obtain the Eulerian circuit.

4.5 Execution Result

The program was executed using the given sample input. The output below shows the Eulerian circuit generated by Iterative Euler Tour Algorithm .

```
1 3
2 3
2 4
2 6
3 5
3 6
4 5
1 3 6 2 4 5 3 2 1
```

Submission details

Task:	Mail Delivery
Sender:	ANGINN
Submission time:	2026-09-22 17:09:32 +0300
Language:	Python3 (PyPy3)
Status:	READY
Result:	ACCEPTED

Test results ▲

test	verdict	time	
#1	ACCEPTED	0.04 s	»»
#2	ACCEPTED	0.04 s	»»
#3	ACCEPTED	0.04 s	»»
#4	ACCEPTED	0.04 s	»»
#5	ACCEPTED	0.04 s	»»
#6	ACCEPTED	0.52 s	»»
#7	ACCEPTED	0.53 s	»»
#8	ACCEPTED	0.53 s	»»
#9	ACCEPTED	0.49 s	»»
#10	ACCEPTED	0.28 s	»»
#11	ACCEPTED	0.04 s	»»
#12	ACCEPTED	0.04 s	»»
#13	ACCEPTED	0.23 s	»»
#14	ACCEPTED	0.21 s	»»
#15	ACCEPTED	0.21 s	»»
#16	ACCEPTED	0.04 s	»»

Code ▲

5. Algorithm Comparison

Algorithm	Approach
Fleury	Builds an Eulerian circuit by avoiding bridge edges whenever an alternative edge is available.
Hierholzer	Builds an Eulerian circuit by continuously following unused edges , then backtracking and merging sub-circuits.
Iterative Euler Tour	Uses an explicit stack to traverse unused edges and construct the final path.

6. Conclusion

Based on the testing results for the CSES Mail Delivery problem, all three algorithms successfully found a valid Eulerian circuit, but with varying levels of efficiency. Fleury's Algorithm proved to be less optimal and encountered a *Time Limit Exceeded* status in several test cases due to the high execution time required for repeated bridge detection. In contrast, Hierholzer's Algorithm and the Iterative Euler Tour Algorithm were highly efficient and successfully achieved an *Accepted* status across all test cases. Hierholzer's Algorithm performed faster by continuously following unused edges without needing to check for bridges, while the Iterative Euler Tour proved highly optimal for large graphs by utilizing an explicit stack to avoid recursion-depth limitations. Overall, Hierholzer's Algorithm and the Iterative Euler Tour Algorithm serve as significantly more efficient and optimal approaches compared to Fleury's Algorithm for solving this problem.

7. GitHub Repository

Repository: <https://github.com/Gaerowes/Graph-Teory-Assignment3-Group-3>

8. AI Usage Disclosure

Gpt: prompt "buatkan template DOCS tugas ini"